

PERSONAL UPGRADE PLAN

Claude Code — 10 Days, 10 New Habits

Tailored to the Contracts ERP workflow (ASP.NET Core + React, Clean Architecture, OFBiz-style domain), built from the 4-hour course transcript and from how you actually use the CLI today.

Prepared: 19 September 2026 · **Claude Code version on this Mac:** 2.1.278

Sources: `Claude-Course.md` (35 chapters), official docs at `code.claude.com/docs` (memory, hooks, skills, commands — checked today), your `~/.claude` + project configuration, and 2,131 prompts of command history.

Time budget: 30–45 minutes per day, always on real backlog work — never on toy examples.

Structure: Part I must-know concepts · Part II file locations · Part III where you stand today · Part IV speed and session length · Part V the ten days · Appendices (cheat sheet, housekeeping flags, course claims to treat with care).

Contents

1. **Part I — Must-know concepts** — the 10 ideas everything else depends on
2. **Part II — File locations** — every path that matters, with its status in your setup
3. **Part III — Where you stand today** — usage snapshot, constraints, and five quick wins
4. **Part IV — Two questions you asked** — A. making responses faster · B. long sessions vs. fresh sessions
5. **Part V — The ten days**
 1. Day 1 — See what you are paying for (/context, /usage, status line, permission cleanup)
 2. Day 2 — Restructure the instruction layers (guardrails first, .claude/rules, personal + local CLAUDE.md)
 3. Day 3 — Plan mode and checkpoints (/plan on a Phase-Two item, /rewind, /diff, /btw)
 4. Day 4 — Context discipline (/compact with instructions, /branch, /fork, handoff notes)
 5. Day 5 — Your first skill (turn a repeated workflow into .claude/skills)
 6. Day 6 — Hooks as hard guardrails (a chime on Stop, a PreToolUse guard for builds and migrations)
 7. Day 7 — Fresh-eyes review (/code-review, /security-review, /simplify on the unbuilt working tree)
 8. Day 8 — Subagents on purpose (.claude/agents: db-analyst, code-reviewer; explicit invocation)
 9. Day 9 — Close the verification loop in the browser (MCP reconnect, live checks, PDF visual QA)
 10. Day 10 — Automate and retrospect (claude -p, /loop, routines, worktrees, /insights, keep/drop)
6. **Appendix A — Command and key cheat sheet**
7. **Appendix B — Housekeeping flags found while preparing this plan**
8. **Appendix C — Course claims to treat with care**

How to use this document. Each day has a *Goal*, a *Why it matters for you*, numbered *Try today* steps with exact commands, a *Done when* check, and a *Keep or drop* prompt. Do the days in order — Day 2 depends on Day 1's measurements, Day 6 protects what Day 5 automates, and Day 10 decides what survives. If a day runs long, stop at the "Done when" line and continue tomorrow; do not skip the check.

Part I — Must-know concepts

Ten ideas, each in a few lines. Everything in the ten days is an application of one of these.

1. The context window is the whole game

Every turn Claude sees one prompt assembled from: system prompt → built-in tool definitions (~17K tokens) → MCP tool definitions → your CLAUDE.md layers and rules → auto-memory → skill descriptions → the conversation. Quality falls and cost rises as this grows; the model remembers the start and the end of a prompt best (primacy / recency), so guardrails belong at the top of CLAUDE.md. `/context` shows the exact breakdown; you have never run it.

2. Instruction layers and load order

Loaded at launch and *concatenated* (not overridden), in this order: `managed/enterprise` → `~/.claude/CLAUDE.md` (you, all projects) → every `CLAUDE.md` from the filesystem root down to the working directory → `CLAUDE.local.md` after its sibling. Files in *subdirectories* (your `client-app/CLAUDE.md`) load on demand when Claude reads a file there. `.claude/rules/*.md` splits a long file into topics; a rule with a `paths:` glob loads only when matching files are touched. `@path` imports pull other files in at launch.

3. CLAUDE.md versus auto-memory — and why neither is enforcement

CLAUDE.md is what *you* write to steer Claude. Auto-memory (`~/.claude/projects/<project>/memory/`, indexed by `MEMORY.md`) is what *Claude* writes for itself: user, feedback, project and reference notes. Both are read as context and can be forgotten under pressure. Anything that must never happen — `dotnet ef database update`, `dotnet build` — needs a **PreToolUse hook**, which is code, not persuasion (Day 6).

4. The extension mechanisms and what each costs

Mechanism	Use it when	Context cost
Rules	Standing conventions; split by topic; scope with <code>paths:</code>	Loaded at launch (or on path match)
Skills	A repeatable <i>procedure</i> with steps and scripts; invoked by <code>/name</code> or auto	Description only (~60 tokens) until invoked
Subagents	Work that should run in its own context: research, zero-bias review, parallel chunks	Own window; only the summary returns
Hooks	Something must happen or be blocked on every event, deterministically	Zero until fired
MCP servers	Quick access to an external system (browser, SaaS)	All tool definitions up front — often 5–20K tokens

The course's most practical rule: prove a workflow with an MCP in five minutes, then convert it into a skill that calls the API or a script directly.

5. Permission modes, and the one you are in

Modes cycle with `Shift+Tab`: default (ask before edits) → accept-edits → plan (read-only exploration) → bypass. Your user settings set `defaultMode: auto`, where the harness allows/denies calls automatically and asks only for risky ones; `/permissions` shows the allow/ask/deny rules. Your project `settings.local.json` has grown to ~15 KB of one-off allow rules — Day 1 cleans it.

6. Plan → build → verify, with checkpoints

Plan mode researches and writes a spec before touching files; "a minute of planning saves ten of building" and cuts tokens because Claude stops exploring alternatives mid-build. Every file edit is checkpointed: `Esc Esc` or `/rewind` restores code and/or conversation. The verify step is what most people skip — tests, a browser screenshot, a rendered PDF, a diff review — and it is where Claude's speed pays off: 80% → 95% → 99% in minutes.

7. Compaction and session lifecycle

Auto-compact fires near the limit; `/compact <what to keep>` lets you steer the summary. `/clear` for unrelated work (you do this 139 times — good). `/resume` and `/rename` for named threads (122 resumes — good). New for you:

`/branch` (fork the conversation in place), `/fork` (copy to a background session), `/btw` (side question that does not enter history), `/export`.

8. Models, effort, thinking

`/model` switches model; `/effort` sets reasoning depth. Extended thinking is billed but *not kept* in the conversation, so "reason as much as you like, answer tersely" is cheap. Sonnet for high-volume, low-judgement work (log reading, data pulls); Opus for design and synthesis. Fast mode (`/fast`) trades price for latency.

9. The parallelism ladder

Single session → subagents (own context, summary back to parent) → agent teams (full independent instances that message each other; roughly 7× tokens) → git worktrees (separate folders, no file collisions, merge at the end). Probabilities multiply: ten subagents at 95% each succeed together only 59% of the time — keep task definitions tiny and return summaries, not raw data. Your `CLAUDE.md` forbids *unrequested* subagents; the ladder is something *you* climb deliberately (Day 8).

10. Security realities

Secrets do not belong in committed instruction files (your dev login is in `CLAUDE.md`; a GitHub token sits in `important-commands.txt` — Appendix B). Bypass mode plus `rm -rf` is a real failure mode; a deny rule or hook removes it. Anything vibe-coded that handles credentials or money needs a human review before it is exposed.

Part II — File locations

Verified against the docs on 19 Sep 2026 and against this machine. "Status" is what exists in your setup right now.

Path	What it is	Scope	Status
<code>~/.claude/settings.json</code>	Your user settings: <code>defaultMode: auto</code> , model opus, voice, theme. Also hooks and permissions at user level.	All projects	exists
<code>~/.claude/CLAUDE.md</code>	Personal instructions for every project (tone, language, reporting habits).	All projects	missing Day 2
<code>~/.claude/rules/*.md</code>	Personal rules applied everywhere.	All projects	missing
<code>~/.claude/skills/<name>/SKILL.md</code>	Personal skills (only a <code>synced/</code> folder from <code>claude.ai</code> exists).	All projects	none of yours Day 5
<code>~/.claude/agents/*.md</code>	Personal subagent definitions.	All projects	missing
<code>~/.claude/projects/~Users~--Contracts/memory/</code>	Auto-memory for this project; <code>MEMORY.md</code> is the index loaded each session. 24 notes today.	This project	active
<code>~/.claude/plans/</code>	Plan-mode documents (one exists — plan mode used about once).	All projects	1 file
<code>~/.claude/history.jsonl</code>	Your prompt history (2,131 entries) — the source of Part III.	All projects	exists
<code>~/.claude.json</code>	Global MCP servers and per-project state; holds the <code>chrome-devtools</code> MCP entry.	All projects	exists
<code>./CLAUDE.md</code>	Project instructions (87 lines): architecture, build/migration rules, dev login.	Repo, committed	exists
<code>./client-app/CLAUDE.md</code>	Frontend conventions (238 lines); loads on demand when files under <code>client-app/</code> are read.	Repo, committed	exists

<code>./CLAUDE.local.md</code>	Private, per-project notes (credentials, local URLs). Must be gitignored — it currently is not .	You, this repo	missing Day 2
<code>./.claude/settings.json</code>	Shared project settings: permissions and hooks the whole team gets.	Repo, committed	missing Day 6
<code>./.claude/settings.local.json</code>	Your private project settings — 14.7 KB of accumulated allow rules.	You, gitignored	bloated Day 1
<code>./.claude/rules/*.md</code>	Topic rules; optional <code>paths:</code> frontmatter for path-scoped loading.	Repo	missing Day 2
<code>./.claude/skills/<name>/SKILL.md (+ scripts/)</code>	Project skills; invoked as <code>/name</code> .	Repo	missing Day 5
<code>./.claude/agents/*.md</code>	Project subagents; manage with <code>/agents</code> .	Repo	missing Day 8
<code>./.claude/hooks/*.sh</code>	Convention for hook scripts referenced from settings via <code>\$CLAUDE_PROJECT_DIR</code> .	Repo	missing Day 6
<code>./mcp.json</code>	Project-scoped MCP servers (shareable). Not used; your MCP lives in <code>~/.claude.json</code> .	Repo	missing
<code>~/.claude/keybindings.json</code>	Custom shortcuts (<code>/keybindings</code>).	All projects	default

Precedence in one sentence. Nothing overrides anything: all discovered instruction files are concatenated, root-most first, so the file closest to where you launched Claude is read *last*; `CLAUDE.local.md` follows its sibling. If two rules contradict, Claude may pick either — prune conflicts rather than relying on order.

Part III — Where you stand today

What you use

- `/clear` ×139 and `/resume` ×122 — disciplined thread management already.
- `/model` ×20, `/compact` ×8, `/rename` ×8, `/effort` ×1.
- Auto mode as default; Opus; voice input in hold mode (the course's "voice dump" habit, already yours).
- Auto-memory working hard: 24 project notes (payroll rounding, GL diagnostics, deploy recipes...).
- Two-level CLAUDE.md (root + `client-app/`) with clear NEVER rules for builds and migrations.
- Chrome DevTools MCP configured (failed to connect this session) and the Claude-in-Chrome extension available.
- Built-in skills present but unused: `/code-review`, `/simplify`, `/security-review`, `/fewer-permission-prompts`.

What you have never touched

- `/context`, `/usage`, `/statusline`, `/insights` — you have no measurement of what a session costs before you type.
- `/plan` (one plan file ever), `/rewind`, `/diff`, `/btw`, `/branch`, `/fork`.
- Rules, skills of your own, agents, hooks — the whole `.claude/` toolbox is empty.
- `/permissions` / `/hooks` / `/agents` / `/mcp` / `/memory` management screens.
- Headless `claude -p`, `/loop`, scheduled routines, worktrees.

Constraints the plan respects

- Claude never builds, runs, or applies migrations — you do. (Day 6 makes this mechanical.)
- No unrequested subagents; sequential work. (Day 8 is explicit invocation only.)
- Auditor rules: financial artifacts are voided and reversed, never hard-deleted.

Five quick wins you can bank on Day 1

1. Run `/context` once in a fresh session and write the baseline number on this page: _____ tokens before the first message.
2. Move the "NEVER" build/migration block to the top of `CLAUDE.md` (primacy effect).
3. Add `CLAUDE.local.md` to `.gitignore` and move the dev login out of the committed `CLAUDE.md` into it.
4. Rotate the GitHub token in `important-commands.txt` and delete the line (Appendix B).
5. Type `/btw` the next time you want a side answer without polluting the thread.

About the presenter's advanced course. The transcript only says it covers "agentic workflows" — his name for skills-plus-scripts, parallelised with subagents and exposed as URLs. This plan covers that territory on Days 5, 8 and 10, adapted to an ERP codebase rather than lead scraping. Nothing here claims to reproduce a syllabus that was not in the text.

Part IV — Two questions you asked after Day 1

Both answers come from the official cost, fast-mode and prompt-caching pages (checked 19 Sep 2026) plus what your history shows. They feed directly into Days 1 and 4.

A. How to make Claude respond faster

Latency has four components: how much Claude must *read* before answering, how much it must *explore* to find the right files, how long it *thinks*, and how long *you* wait between turns. Fast mode only touches the third; the other three are usually the bigger win and free.

Lever	What to do	Effect
1. Shrink what it reads	Every request re-sends the whole conversation, and a tool-heavy turn is several requests. Keep sessions scoped (section B), keep <code>CLAUDE.md</code> under ~200 lines, move procedures into skills, disable MCP servers you are not using today (<code>/mcp</code>), prefer CLI tools (<code>gh</code> , <code>mysql</code> , <code>dotnet ef ... script</code>) over MCP wrappers.	Largest and free. Time-to-first-token grows with uncached context; a 30K-token prompt answers noticeably faster than a 200K one.
2. Shrink the exploration	Name files: <code>@Application/Accounting/Services/GeneralLedgerService.cs</code> instead of "the GL service". Give the error text, the line, the expected value. Use plan mode for multi-layer work so the search happens once. Install a code-intelligence (LSP) plugin for C# and TypeScript from <code>/plugin</code> — one "go to definition" replaces a grep plus three speculative file reads, and type errors are reported after each edit without a build.	Second largest. Most slow turns in your history are Claude searching, not Claude typing.
3. Shrink the thinking	<code>/effort low</code> or <code>medium</code> for mechanical work (renames, DTO fields, label changes, applying a plan you already approved); <code>high / xhigh</code> for design and ledger logic. Thinking tokens are billed as output and are the default's biggest hidden latency.	Immediate. Pair with lever 4 for the fastest configuration on simple tasks.
4. Fast mode <code>/fast</code>	Same Opus model, up to 2.5× faster output, priced at \$10 / \$50 per MTok (flat across the 1M window). Opus 5 and 4.8 only; toggling it on switches you to Opus if needed. On Pro/Max it bills to usage credits even while plan usage remains, so credits must be enabled first (<code>/usage-credits</code>). Turn it on at the <i>start</i> of a session: the first fast request re-caches the entire conversation at the fast uncached price, so enabling it three hours in is the expensive way. It falls back to normal speed automatically when its own rate limit is hit (<code>⚡</code> turns grey).	Use for live debugging and rapid UI iteration with the dev server running. Leave it off for long autonomous runs, reviews and reports.
5. Match the model	Sonnet for high-volume, low-judgement turns (log digests, data pulls, bulk edits from a spec); <code>model: haiku</code> or <code>sonnet</code> in subagents. Opus where the judgement is the product.	Faster and cheaper per token; keeps Opus turns short.
6. Shrink your own waiting	Press <code>Esc</code> the moment it heads the wrong way, then <code>/rewind</code> . Queue the next message with <code>Enter</code> while it works. Let auto mode and pattern allow-rules handle permissions (each prompt is latency <i>you</i> add). Put the Stop chime in (Day 6) and stop watching. Send long research to the background with <code>/fork</code> .	Often the biggest wall-clock win in interactive work.
7. Shrink the output	In <code>~/ .claude/CLAUDE.md</code> : "Answer tersely; show the diff, not a narrative; explain only when asked." Output tokens are the slowest tokens.	Small but constant.
8. Keep the cache warm	The prompt cache lives 1 hour on a subscription (5 minutes once you are drawing on usage credits). A first message after a longer break re-processes the whole context — slow and full price. Changing <code>CLAUDE.md</code> , rules, MCP servers or the model mid-session also invalidates it. <code>/usage</code> shows a <i>Prompt cache</i> line with the hit rate and a likely cause for the last miss.	Explains the "why is the first answer after lunch so slow" effect.

Symptom → **fix**. *Slow before the first token*: context too big or cache cold → lever 1 / 8, or start a new session. *Many tool calls before any edit*: vague prompt or no LSP → lever 2. *Long "thinking" on a trivial change*: lever 3. *Typing is slow once it starts*: lever 4 or 7. *You are waiting on permission prompts*: lever 6.

B. Long sessions: what you get, what it costs, and the practice that replaces it

WHAT YOUR BELIEF GETS RIGHT

The conversation is the only place the last three hours of reasoning live. Nothing else knows that the void-payment path was rewritten twice or why you rejected the first Telerik approach. Continuity is real.

WHAT IT GETS WRONG — FROM THE DOCS AND FROM YOUR HISTORY

1. **Every turn re-sends everything.** A one-line question in a session that has been open all day carries the whole conversation, and each tool batch inside a turn is another request carrying it again. Cached tokens are cheaper, not free, and they still take time to read.
2. **The continuity is already lossy.** When the window fills, auto-compaction replaces the history with a summary. After the first compaction you no longer have "the context" — you have Claude's précis of it, chosen without your priorities. And compacting a large session is itself a large request.
3. **Quality degrades before cost does.** Rules from `CLAUDE.md` sit at the far end of a 300K prompt; three abandoned approaches remain in view and pull later answers toward them; a fix from hour one is re-broken in hour four because the instruction that motivated it is buried. This is what the course calls context rot, and the docs' advice is blunt: "unexpectedly high spend usually traces back to long sessions that were never cleared."
4. **Breaks punish long sessions most.** The cache expires after an hour (five minutes on usage credits). Coming back to a huge session after lunch means re-processing all of it at full price and full latency. Pro/Max users are offered *resume from a summary* for exactly this reason.
5. **The durable context survives `/clear`.** `CLAUDE.md`, rules, auto-memory, plan files, the git diff and a handoff note are all still there. A fresh session that reads a 15-line handoff plus `git diff` usually outperforms the 300K-token session it replaces — and answers faster (section A, lever 1).

THE PRACTICE: ONE SESSION = ONE TASK WITH A DEFINITION OF DONE

Situation	Do
Starting a feature, bug or report	New session. Brief it (or <code>/plan</code>), state the definition of done, name the files.
Follow-up on work just finished, same files, same goal	Stay. This is what continuity is for.
Reached a milestone ("it works, now polishing")	<code>/compact</code> Keep: files changed and why, open bugs with exact errors, decisions taken. Drop: tool output.
A tangent or a "while I'm here" question	<code>/btw</code> for a side answer; <code>/branch</code> if the tangent becomes real work.
An approach failed and you are abandoning it	<code>/rewind</code> and restore <i>conversation and code</i> to before it — the failed attempt leaves the context instead of anchoring the next one.
Switching subject (payroll → Telerik → test environment)	Handoff → <code>/rename</code> → <code>/clear</code> . Always.
Build/debug cycle done, feature committed	Handoff → <code>/rename</code> → <code>/clear</code> . Do not reuse the thread for the next item.
Context above ~60%, or any compaction has happened	Finish the current step, then handoff and clear. The status line from Day 1 makes this visible.
Claude repeats a mistake it already fixed, or forgets a rule	That is context rot, not a model problem. Clear now; add the rule to <code>.claude/rules/</code> first.
Coming back after more than an hour	Accept the resume-from-summary offer, or <code>/resume</code> the named thread and start with "read your handoff".
A verbose sub-task (read 5,000 log lines, compare 3 SQL outputs)	Ask explicitly for a subagent or the <code>db-analyst</code> agent (Day 8) so the noise stays out of your window.

THE HANDOFF, SO CLEARING COSTS NOTHING

```
Before /clear: "Write a handoff for a fresh session: goal · what is done · what is next · open errors
(verbatim) ·
                decisions and why · files touched. Save anything non-obvious and durable to memory."
Then:          /rename <task-name> → /clear
Fresh session: "Read the handoff in memory for <task-name>, run git diff --stat, then continue with
step N."
```

Add a `# Compact instructions` section to `CLAUDE.md` ("preserve exact error text, file list, and decisions; drop tool output") so even automatic compaction keeps what you care about.

Applied to your recent work. The KendoReact v16 migration (152 files) and the project-report rebuild each lived in long threads. The cheaper shape: one session per batch of files or per plan step, a `MIGRATION-STATUS.md` or plan file as the shared state, a handoff at each batch boundary. Same continuity, a fraction of the tokens, and each session starts at the fast end of the latency curve.

Rule of thumb. If you would need more than two sentences to tell a colleague what this session is about, it is two sessions. If a fresh session with the handoff could not continue the work, the handoff — not the session — is what needs fixing.

1 See what you are paying for

~35 min

GOAL

Measure your context and cost baseline, put a live token readout on screen, and shrink the permission allow-list from one-off commands to patterns. (Part IV-A explains why this is also the first speed fix.)

WHY IT MATTERS FOR YOU

You run long, heavy sessions (Telerik report pipeline, GL reconciliation, KendoReact migration across 152 files) on Opus. Without `/context` you cannot tell whether a slow, drifting session is the task or the 40K tokens of overhead you carry into it. The bloated `settings.local.json` is also a signal: every one-off `awk` command you approved forever is a rule Claude now reads and that prompts you again the next time the command differs by one character.

TRY TODAY

1. Open a fresh session in the repo and type `/context` before anything else. Note the split: system prompt · system tools · MCP tools · memory files · skills · messages · free space. Write the total on Part III.
2. Type `/usage` (alias `/cost`) and `/status`. Then `/insights` once — it generates an HTML report of how you actually work; skim it for the tasks you repeat.
3. Type `/statusline` and ask for: model · context-used bar with percentage · git branch · cost. This is terminal-only and permanent.
4. Run `/fewer-permission-prompts`. It scans your transcripts and proposes a prioritised allow-list. Accept the pattern rules (`Bash(mysql *)`, `Bash(git diff *)`, `Bash(python3 *)` ...) and then open `.claude/settings.local.json` and delete the literal one-off entries (the pdfplumber one-liners, the specific `awk` line-number commands, the CRUX curl).
5. Add explicit **deny** rules while you are there — these are enforced even in bypass mode:

```
{
  "permissions": {
    "deny": [
      "Bash(dotnet ef database update*)",
      "Bash(dotnet ef migrations remove*)",
      "Bash(rm -rf *)",
      "Read(./**/appsettings.Production.json)"
    ]
  }
}
```

6. Type `/mcp`. If `chrome-devtools` shows disconnected (it did today), reconnect it now so Day 9 is not blocked; if it will not connect, note it and plan to use the Chrome extension instead (`/chrome`).

DONE WHEN

You know your pre-message token overhead to the nearest thousand, a status line shows context percentage, and `settings.local.json` is under 3 KB with four deny rules.

KEEP OR DROP

The status line and deny rules stay forever. `/insights` is a monthly habit, not a daily one.

2 Restructure the instruction layers

~45 min

GOAL

Guardrails at the top, secrets out, one file per topic, a personal layer for things that are about *you* rather than this repo.

WHY IT MATTERS FOR YOU

Your root `CLAUDE.md` is good but flat: the NEVER rules sit two-thirds of the way down, the dev password is committed, and the frontend file is a 238-line monolith that loads whole whenever a `.tsx` file is read. Path-scoped rules give you the same steering at lower cost, and a `~/.claude/CLAUDE.md` means your preferences (Arabic-capable reports, no builds by Claude, concise answers) follow you into the Power BI folder, the OFBiz reference folder, and the test-environment work.

TRY TODAY

1. In `CLAUDE.md`, move "Database Migrations" and "Build Rules" into a single **Hard rules** block directly under the title. Keep each rule to one line.
2. Create `CLAUDE.local.md` with the dev login and any local URLs; delete them from `CLAUDE.md`; add `CLAUDE.local.md` to `.gitignore` (it is *not* ignored today — `git check-ignore -v CLAUDE.local.md` returns nothing).
3. Create `.claude/rules/` and split, with path scoping:

```
.claude/rules/
backend-cqrs.md      # paths: ["Application/**/*.cs", "API/**/*.cs"]
persistence.md       # paths: ["Persistence/**/*.cs", "Domain/**/*.cs"]
reporting-telerik.md # paths: ["API/Reporting/**"]
frontend-react.md    # paths: ["client-app/src/**/*.{ts,tsx}"]
accounting-rules.md  # no paths: soft-delete/void, payroll rounding, GL invariants
```

Frontmatter example:

```
---
paths:
  - "API/Reporting/**"
---
# Telerik Reporting
- Code-defined reports, literal TextBox per cell. Never Telerik.Reporting.List / Fields.X binding
  (renders zero rows).
- Arabic text: reuse the payment-voucher pipeline; verify output visually (Day 9).
```

Ask Claude to do the split: *"Split CLAUDE.md and client-app/CLAUDE.md into .claude/rules files using the rules spec with paths frontmatter; keep every rule, remove duplicates, do not invent new ones."* Then read the result yourself.

4. Create `~/.claude/CLAUDE.md` (≤ 25 lines): who you are, language expectations, "report outcomes plainly", "prefer skills over MCP", "when a mistake repeats twice, propose a CLAUDE.md or rule line".
5. Verify: `/memory` lists the files; `/context` shows the rules under memory files; open a `.tsx` file in conversation and confirm the frontend rule appears only then.

DONE WHEN

Root `CLAUDE.md` is under 60 lines with hard rules on top, no credentials are tracked by git, and `/context` at launch is lower than yesterday's baseline.

KEEP OR DROP

Treat the rules like code: when Claude keeps making the same mistake, say *"add this to the relevant rule so a fresh session gets it right"* — the course's favourite instruction, and it maps onto your auto-memory habit.

3 Plan mode and checkpoints

~45 min

GOAL

Do one real Phase-Two item the plan-first way, and learn to undo with `/rewind` instead of asking Claude to revert.

WHY IT MATTERS FOR YOU

Your biggest changes lately (project-report in-app view, Telerik integration, soft-delete plan) were written straight into the working tree and are still unbuilt, with a "build-fix round" expected. Plan mode front-loads the research — which handlers, which OData sets to register, which auditor rule applies — into a document you approve before any file changes, and it is read-only, so it cannot violate your build rules by accident. Checkpoints let you try an approach and roll it back in seconds without a git stash.

TRY TODAY

1. Pick one item from `second phase/2nd-phase-ERP-SYS-transcription.md` that is small but real (a new field, a new list view, a status rule).
2. Type `/plan` followed by the item in your own words (voice-dump is fine — plan mode is where verbosity is cheapest). Let it explore; answer its *AskUserQuestion* prompts.
3. Read the plan critically: does it name the CQRS file (`VerbEntity.cs`), the DTO, the OData registration in `GetEdmModel()`, the RTK Query slice, the RBAC/status-ribbon pattern from your frontend conventions? If not, say so — edits are cheap here and expensive later.
4. Approve and let it build. Then, deliberately: ask for a small extra change you do not want, press `Esc Esc` (or `/rewind`), choose *restore code*, and confirm the file is back.
5. Type `/diff` to review the working-tree changes in the terminal before you build. Use `/btw` for the side question you would normally have asked inline ("does OFBiz do this with a status item?") — it is answered without entering history.
6. Build and debug yourself as usual. When something fails, paste the error and let the same session fix it — the plan is still in context.

DONE WHEN

One backlog item shipped with a plan file in `~/.claude/plans/`, one rewind performed, and you used `/diff` and `/btw` at least once each.

KEEP OR DROP

Rule of thumb from the course, adjusted for you: plan mode for anything touching more than two layers (domain + handler + controller + UI) or anything with ledger side-effects. Skip it for one-file fixes.

4 Context discipline

~30 min

GOAL

Replace "clear and re-explain" with steered compaction, conversation branching, and written handoffs — the session practice from Part IV-B, made into a routine.

WHY IT MATTERS FOR YOU

139 clears and 122 resumes say you juggle many threads; 8 compacts say you usually let auto-compact decide what survives. Auto-compact does not know that the PAYROL_PAYMENT rounding rule or the 78,990 date-typo finding is the one fact that must not be lost. Steered compaction and handoff notes keep the expensive part of a session — the understanding — while dropping the tool noise.

TRY TODAY

1. In a long-running session, before it hits the buffer, type:

```
/compact Keep: the list of files changed and why, open bugs with their exact error text, and the decisions we made about void-vs-delete. Drop: tool output, file contents we already edited.
```

2. Check `/autocompact` and leave it on auto unless you see compaction firing mid-task; then raise the threshold.
3. When a session forks into two directions (e.g. "fix the report" and "why does the trial balance differ"), type `/branch trial-balance` to continue the second in a copy while the first stays intact. Use `/fork` when the second thread should run in the background instead.
4. Before any `/clear` at the end of a work block, say: "Write a handoff: current state, next step, gotchas — save anything non-obvious to memory." Your auto-memory already stores project notes; this makes the save deliberate. Then `/rename` the session so `/resume` finds it.
5. Try the cheap-transcript trick once: dictate a long request, paste it to a Sonnet session (`/model sonnet`) with "compress this into a precise task statement", then give the compressed version to your Opus session.
6. Use `/export handoff-2026-09-22.md` at the end and skim it: that is what compaction has to summarise.

DONE WHEN

You have one steered compaction, one branch, and one handoff note in memory that a fresh session can pick up without re-reading the code.

KEEP OR DROP

Keep the handoff habit unconditionally. Branch/fork only when you actually feel the two threads fighting.

5 Your first skill

~45 min

GOAL

Turn a workflow you have repeated in chat at least three times into a `.claude/skills/` folder with a checklist and, where it helps, a script.

WHY IT MATTERS FOR YOU

Your memory index is a list of skill candidates: verifying a GL balance three different ways, rendering a real-data report to PDF with headless Chrome, generating a migration and its SQL script for review, looking up a Phase-Two item by number. Each time you re-explain these you spend tokens and risk drift. A skill costs ~60 tokens until invoked, is version-controlled, and — per the course — gets *better* because Claude updates it when a step fails.

TRY TODAY

1. Pick the first: **GL account check**. Create `.claude/skills/gl-check/SKILL.md`:

```
---
name: gl-check
description: Verify a GL account's balance in the local erp_contracts MySQL DB the three ways this
  codebase computes it (ACCTG_TRANS_ENTRY sum, trial-balance query, project GL view) and report
  differences.
  Use when asked to check, reconcile, or explain a GL/bank/trial-balance figure.
arguments: [account, period]
allowed-tools: Bash(mysql *) Read Grep
---
Account: $account Period: $period (YYYY-MM; default = current month)

1. Read the SQL in `scripts/gl_check.sql`; substitute the account and period.
2. Run it read-only against the dev DB (credentials come from CLAUDE.local.md, never from this
  file).
3. Report a 3-row table: method · balance · entry count, then the differences and the most likely
  cause
  (see memory notes on PAYROL_PAYMENT filtering and per-invoice rounding).
4. If a step fails, fix the script here so the next run does not repeat the failure.
```

2. Have Claude write `scripts/gl_check.sql` from the three implementations it already knows (GeneralLedgerService, the trial-balance query, the project report). Review the SQL yourself.
3. Start a **new** session (skills register at launch) and type `/gl-check 110100 2026-08`. Compare with the figures from the August bank-balance investigation you already resolved.
4. Second skill, 10 minutes: `migration` with `disable-model-invocation: true` and `allowed-tools: Bash(dotnet ef migrations add *) Bash(dotnet ef migrations script *)`, whose steps are: add the migration, generate the SQL script to `/tmp`, print the diff of the migration file, **stop** and remind that applying is manual.
5. Third, if time: `report-pdf` wrapping the HTML → headless Chrome → pypdfium2 QA recipe from your memory notes, with `!\`ls "$CLAUDE_PROJECT_DIR"/*.html`` injected so it sees candidate sources.

DONE WHEN

`/gl-check` works from a fresh session and its output matches a number you already trust; `/migration` refuses to apply anything.

KEEP OR DROP

Add a skill only after the third manual repetition. Skills that describe a *convention* rather than a *procedure* belong in rules instead.

6 Hooks as hard guardrails

~40 min

GOAL

A chime when Claude stops or needs you; a PreToolUse guard that makes your build and migration rules impossible to break; a log of what got blocked.

WHY IT MATTERS FOR YOU

Today "NEVER run dotnet build" is a sentence in a markdown file. It has held, but it is context, not policy, and it competes with everything else in the window after a compaction. A hook is a shell script that runs before every Bash call and exits 2 to block — no model judgement involved. The chime is the course presenter's single most-used trick for running several sessions without idling any of them.

TRY TODAY

1. Create `.claude/hooks/guard-bash.sh` (no `jq` on this Mac, so Python parses stdin):

```
#!/bin/bash
cmd=$(python3 -c 'import sys,json; print(json.load(sys.stdin).get("tool_input",
{}).get("command",""))')
if echo "$cmd" | grep -Eq 'dotnet +(build|run|watch)|dotnet ef database update|dotnet ef migrations
remove|npm run (build|start)|yarn (build|start)|rm -rf'; then
  echo "BLOCKED by .claude/hooks/guard-bash.sh: '$cmd' - CLAUDE.md hard rule (builds/migrations are
manual)." >&2
  echo "$(date '+%F %T') $cmd" >> "$CLAUDE_PROJECT_DIR/.claude/hooks/blocked.log"
  exit 2
fi
exit 0
```

`chmod +x .claude/hooks/guard-bash.sh` and add `.claude/hooks/blocked.log` to `.gitignore`.

2. Create `.claude/settings.json` (shared, committed) with the guard, and put the sounds in your user settings so they follow you to every project:

```
// .claude/settings.json
{ "hooks": { "PreToolUse": [ { "matcher": "Bash",
  "hooks": [ { "type": "command", "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/guard-
bash.sh" } ] } ] } }

// ~/.claude/settings.json (merge into the existing object)
{ "hooks": {
  "Stop": [ { "hooks": [ { "type": "command", "command": "afplay
/System/Library/Sounds/Glass.aiff" } ] } ],
  "Notification": [ { "hooks": [ { "type": "command",
    "command": "osascript -e 'display notification \"Claude needs input\" with title \"Claude
Code\"'" } ] } ]
} }
```

3. Restart the session, type `/hooks` and confirm all three appear with their source file.
4. Test the guard on purpose: `"run dotnet build to check"`. Expect the block message in the conversation and a line in `blocked.log`. Test the chime by asking any question.
5. Optional: a `PreCompact` hook that appends `git status --short` to a scratch file, so a post-compaction session can recover which files were in flight.

DONE WHEN

A build attempt is blocked mechanically, the chime plays on Stop, and `.claude/settings.json` is committed so every clone of the repo gets the guard.

KEEP OR DROP

Keep both. If the chime annoys you in single-session work, keep `Notification` and drop `Stop`.

7 Fresh-eyes review

~40 min

GOAL

Run the built-in review skills on the large, unbuilt working tree before you build it, and learn what a zero-context reviewer catches that the author session does not.

WHY IT MATTERS FOR YOU

The course's reviewer-subagent argument is about bias: a session that just wrote 2,000 lines of Telerik and soft-delete code defends its choices. Your working tree holds exactly that kind of change — project-report PDF export rebuilt on server-side Telerik, void-payment primitives, required-PartyId guards — plus auditor constraints where a mistake is a ledger error, not a UI glitch. `/code-review`, `/security-review` and `/simplify` give you the reviewer without you spawning agents yourself.

TRY TODAY

1. In a **fresh** session (fresh is the point), type `/code-review high`. It reviews the current diff for correctness bugs. Read every finding against your rules: lazy loading disabled → missing `.Include()`; `Result<T>` never throws; OData sets registered; snake_case only on Domain.
2. Type `/security-review`. Pay attention to anything touching `AccountController`, `PaymentHelperService` and the void/reverse paths.
3. Pick one small handler and type `/simplify` — quality only, it applies the cleanups; check the diff with `/diff`.
4. Ask the reviewer's question explicitly once, the way the course frames it: *"You have no context on why this was written this way. Read `Application/.../VoidPayment.cs` and tell me what looks wrong or odd."* Compare with what the author session told you.
5. For findings you disagree with, say why and ask for the rule that would prevent the false positive — add it to `.claude/rules/accounting-rules.md`.
6. Then build, as you always do, and note how many of the build-fix items the review had already flagged.

DONE WHEN

Three review passes run, at least one real finding fixed before the build, and one rule added from a disagreement.

KEEP OR DROP

Make `/code-review` the last step before every build you run manually. `/security-review` whenever payments, auth or user↔party linkage change.

8 Subagents on purpose

~45 min

GOAL

Define two agents in `.claude/agents/`, invoke them explicitly, and see the context-isolation benefit without changing your "no unrequested subagents" rule.

WHY IT MATTERS FOR YOU

Two of the course's three recommended agents fit you exactly. A **db-analyst** on Sonnet can churn through hundreds of rows of ACCTG_TRANS_ENTRY, payroll snapshots or audit logs in its own window and return a 20-line answer — the thing that today floods your Opus session. A **code-reviewer** gives the Day 7 effect on demand. The third (QA/test runner) is deferred because Claude does not build in this repo. Your CLAUDE.md rule stays: the parent only uses them when *you* ask.

TRY TODAY

1. Create `.claude/agents/db-analyst.md`:

```
---
name: db-analyst
description: Read-only analyst for the local erp_contracts MySQL dev DB. Use only when the user
explicitly asks
  for a data reconciliation or a count/sum across many rows. Returns a short summary and the SQL
used.
tools: Bash, Read, Grep
model: sonnet
---
You answer data questions against the local dev database with SELECT statements only (never
INSERT/UPDATE/DELETE/DDDL).
Credentials: read CLAUDE.local.md. Return at most 25 lines: finding · the SQL · caveats.
Known traps: PAYROL_PAYMENT nets are rounded per invoice then summed; aggregate payroll payments
carry no invoice link;
MainRole and PARTY_ROLE can diverge. Never paste raw row dumps back to the caller.
```

2. Create `.claude/agents/code-reviewer.md` (Opus, tools Read/Grep/Glob only, prompt: "You have no context on intent. Review for correctness against the project rules; report findings ranked by severity with file:line; propose no rewrites.").
3. New session. `/agents` should list both. Ask: "Use the db-analyst agent to compare payroll totals for August between PAYMENT rows and the snapshot table." Watch the Task block; note that only the summary lands in your context.
4. Parallel on purpose, once: "Use two db-analyst agents in parallel: one for July, one for August; then compare." Observe that each returns a summary and that the parent does the comparison — the course's "parent synthesises, children fetch" rule.
5. Ask the code-reviewer for one file you changed today, and compare with `/code-review` from Day 7.

DONE WHEN

Both agents appear in `/agents`, one data question was answered by db-analyst with less than 30 lines returned, and you can say in one sentence when you would reach for an agent versus a skill.

KEEP OR DROP

Keep db-analyst. Keep code-reviewer only if it caught something `/code-review` did not. Agent teams (7× tokens) remain off; consider them only for one-off repo-wide audits with a hard budget.

9 Close the verification loop in the browser

~45 min

GOAL

Have Claude verify a UI or report change against the running dev server and against a rendered PDF, instead of you being the only tester.

WHY IT MATTERS FOR YOU

The KendoReact PDF export "worked" until someone looked at the Arabic. The theme visual pass after the v16 upgrade is still open. Your auditor wants the in-app project report as the primary view. All of these are "task → do → **verify**" gaps that a screenshot loop closes cheaply — and you already have both the Chrome DevTools MCP and the Claude-in-Chrome extension; one of them just needs to be working.

TRY TODAY

1. Start the dev server yourself (frontend on :3000, API on :5001 — Claude never starts it). Type `/mcp`; if `chrome-devtools` still fails to connect, use the extension: `/chrome`, and tell Claude to use the Claude-in-Chrome tools.
2. Ask: "Log in to localhost:3000 with the credentials in `CLAUDE.local.md`, open the project report view for project X, take a screenshot, and check: Arabic labels render right-to-left, totals match the Excel export for the same project, no console errors." Have it read `read_console_messages` and network requests, not just the picture.
3. Fix one finding in the same session, then ask it to re-verify — the loop, not the one-shot.
4. PDF: ask it to trigger the server-side Telerik export, save the file, render page 1 to PNG with the `pypdfium2` venv recipe from memory, and compare the Arabic shaping against the payment voucher (your known-good pipeline). Read the PNG yourself too.
5. Record the walkthrough as a GIF with the extension's recorder if you want to send it to the auditor.
6. Write the working recipe into `.claude/skills/ui-verify/SKILL.md`: login → navigate → screenshot → console/network → report — so Day 5's habit compounds.

DONE WHEN

One screen and one PDF verified by Claude with evidence (screenshot + console log + rendered page), and a `ui-verify` skill exists.

KEEP OR DROP

Keep for every frontend or report change. If the MCP keeps disconnecting, drop it and standardise on the extension — one browser tool is enough; two sets of tool definitions is context you pay for.

10 Automate, parallelise, retrospect

~45 min

GOAL

Run Claude without a chat window for one recurring chore, try worktree isolation for one feature, and decide — with data — which of the last nine days' habits survive.

WHY IT MATTERS FOR YOU

The course ends with "put your skills on a URL". For an ERP developer the equivalent is quieter: a headless run that summarises yesterday's Serilog warnings, a routine that checks the prod box, a worktree so the soft-delete rework does not sit half-done next to a hot fix. The last thirty minutes are the most valuable of the ten days: habits you do not need cost context every session.

TRY TODAY

1. Headless, read-only, no build risk:

```
claude -p "Read API/logs/logfile$(date +%Y%m%d).log and the previous day's file. Group WARN/ERROR by source and message template, count them, flag anything new versus the previous day. Output markdown." \
--allowedTools "Read,Grep,Glob" --output-format text > /tmp/log-digest.md
```

Read the digest. If useful, wrap it as a skill and schedule it: `/loop 1d /log-digest` during a working session, or `/schedule` for a cloud routine that does not need your Mac awake.

2. `/goal` once, on something bounded: *"work until /code-review high returns no correctness findings on the current diff"*. Watch it stop on the condition, not on your interruption.
3. Worktree for the soft-delete rework (shared MySQL and ports mean: one dev server at a time):

```
git worktree add ../Contracts-soft-delete -b feature/soft-delete
cd ../Contracts-soft-delete && claude
```

- `CLAUDE.local.md` is per-worktree (gitignored); import a home-directory file if both need the same private notes.
4. Send one long research question (e.g. NPHIES/ZATCA patterns in the EIDD PDF) to the background with `/fork`; `/tasks` shows it.
5. **Retrospective (30 min)**. Run `/insights` again and `/context` in a fresh session. Fill the table below. Delete what you dropped — rules, hooks, agents, MCP servers all cost something.

Habit	Keep?	Evidence (what it saved or caught)
Status line + <code>/context</code> baseline		
Rules split + local/personal CLAUDE.md		
Plan mode for multi-layer changes		
Steered <code>/compact</code> + handoff notes		
<code>/gl-check</code> , <code>/migration</code> , <code>/ui-verify</code> skills		
Guard hook + chime		
<code>/code-review</code> before every build		
db-analyst agent		
Browser verification loop		
Headless digest / routine / worktree		

DONE WHEN

The table is filled, at least two things are deleted, and your launch `/context` is lower than on Day 1 despite everything you added.

Appendix A — Command and key cheat sheet

Measure

/context	What is in the window, by category
/usage · /cost	Tokens and cost this session
/status	Model, mode, account, version
/statusline	Configure the permanent readout
/insights	HTML report of how you work
/doctor	Setup diagnostics

Steer the session

/plan [text]	Enter plan mode (also Shift+Tab)
/compact [instructions]	Summarise with priorities
/autocompact	Set the auto threshold
/clear · /resume · /rename	Thread management
/branch [name]	Fork conversation in place
/fork · /background	Copy / detach to background
/btw	Side question, no history
/rewind · Esc Esc	Restore code and/or conversation
/diff	Review working-tree changes
/export [file]	Dump conversation as text
/goal [condition]	Work until a condition holds
/model · /effort · /fast	Model, reasoning depth, speed

Configure

/memory	Edit CLAUDE.md layers, toggle auto-memory
/permissions	Allow / ask / deny rules
/hooks	View configured hooks and sources
/agents	Manage subagents
/mcp	Reconnect / enable / disable servers
/chrome	Claude-in-Chrome settings
/config · /keybindings	Settings, shortcuts file
/init	Generate or improve CLAUDE.md
/fewer-permission-prompts	Build an allow-list from history

Quality & automation

/code-review [level]	Bug review of the diff or a PR
/security-review	Security pass on the diff
/simplify	Apply quality cleanups
/loop [interval] cmd	Repeat a prompt or skill
/schedule	Cloud routines on a cron
/tasks	Background work and agents
claude -p "..."	Headless one-shot from a shell

Keys and prefixes

Shift+Tab	Cycle permission modes
Esc Esc	Rewind menu
! command	Run a shell command, output lands in the thread
@path	Attach a file or folder to the prompt
Enter while running	Queue the next message

Commands verified against the docs' command list on 19 Sep 2026 for Claude Code 2.1.x. Names change; `/help` is authoritative.

Appendix B — Housekeeping flags found while preparing this plan

#	Finding	Why it matters	Do
1	<code>important-commands.txt</code> (tracked, modified) contains a GitHub personal access token in plain text.	Anyone with the repo history has it; Claude reads the file whenever it greps the root.	Revoke on GitHub, delete the line, consider <code>git filter-repo</code> if the repo is shared.
2	Dev login (email + password) is in the committed <code>CLAUDE.md</code> .	Belongs in <code>CLAUDE.local.md</code> ; the browser-check instruction can stay and point there.	Day 2, step 2.
3	<code>CLAUDE.local.md</code> is not in <code>.gitignore</code> .	The first time you create it, it will be staged.	Add the line now.
4	<code>.claude/settings.local.json</code> is 14.7 KB with dozens of literal one-off commands, including an <code>export MYSQL_PWD=...</code> rule that embeds the password.	Context and prompt noise; secret in a config file.	Day 1, steps 4–5.
5	Repo root holds 18 stray files in git status (screenshots, xlsx, PDFs, an <code>AD</code> added-then-deleted set).	Every <code>ls / glob</code> Claude runs pays for them; the course calls this "workspace bloat".	Move deliverables to a <code>docs/</code> or <code>deliverables/</code> folder; untrack the rest.
6	<code>chrome-devtools</code> MCP failed to connect this session ("Connection closed").	Day 9 depends on a browser tool; two half-working ones cost double context.	<code>/mcp</code> reconnect; else standardise on the extension.
7	No <code>.claude/settings.json</code> (shared) — every safeguard is personal or in prose.	Another clone or the test-environment box gets none of it.	Day 6 creates it.
8	The 380 KB <code>Claude-Course.md</code> transcript is staged at the repo root.	It is source material, not project code, and it inflates every full-text search.	Keep the notes, remove the transcript from the repo (or move under <code>docs/reference/</code>).

Order of operations. Do #1 before anything else today; it takes two minutes and does not depend on the plan.

Appendix C — Course claims to treat with care

Claim in the transcript	What the docs / your setup say
"The global <code>~/.claude</code> supersedes the local <code>.claude</code> ."	Nothing supersedes; files are concatenated root-first, and the closest file is read last. Resolve conflicts by pruning, not by placement.
"Move your CLAUDE.md into <code>.claude/</code> to unlock advanced features."	<code>.claude/CLAUDE.md</code> is a valid location but unlocks nothing; the toolbox is <code>rules/</code> , <code>skills/</code> , <code>agents/</code> , <code>settings.json</code> . Keep yours at the root.
"Memory is a single global <code>memory.md</code> ."	Auto-memory is per project under <code>~/.claude/projects/<project>/memory/</code> with typed notes and a <code>MEMORY.md</code> index — which you already use heavily.
"Bypass permissions is what most people use now."	You are on auto mode, which is the safer middle. Deny rules and hooks apply in every mode; use them rather than trusting prose.
"Plugins will be deprecated into skills."	Speculation. Plugins bundle skills, agents, hooks and MCP servers together; the built-in <code>/code-review</code> and <code>/simplify</code> you will use on Day 7 are plugin-delivered.
"Agent teams cost ~7×; Opus 4.6 has a 200K window."	Order-of-magnitude guidance from early 2026. Check <code>/usage</code> and <code>/model</code> for today's numbers before budgeting anything.
"Subagents' new definitions need a new session."	Consistent with the docs and with skills: register at launch. Plan Days 5 and 8 around a restart.
"MCP servers will auto tool-search when definitions exceed 10% of context."	Deferred tool loading exists (you saw it in this very session), but it does not make a badly designed server free — the skill-instead-of-MCP rule still stands.

Prepared with Claude Code from `Claude-Course.md`, the Claude Code documentation (memory, hooks, skills, commands pages, fetched 2026-09-19), `~/.claude/history.jsonl`, `~/.claude/settings.json`, `.claude/settings.local.json`, the project memory index, and the repository's CLAUDE.md files. Source HTML: `Claude-Code-10-Day-Plan.source.html`.